

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Kalibrácia kamery

Študijný program:	robotika a kybernetika
Študijný odbor:	kybernetika
Školiace pracovisko:	ústav robotiky a kybernetiky
Predmet:	počítačové videnie a spracovanie obrazu
Zadanie vypracoval:	Dominik Kupec a Martin Brandobur

17.3.2026

Bratislava

Obsah

1	Zadanie	3
1.1	Úlohy.....	3
2	Vypracovanie	5
2.1	Kalibrácia kamery	5
2.1.1	Postup riešenia	5
2.1.2	Použité metódy.....	5
2.1.3	Ukážky výsledkov	6
2.1.4	Získaní hodnoty parametrov kamery	7
2.2	Detekcia geometrických tvarov	7
2.2.1	Postup riešenia	7
2.2.2	Použité metódy.....	8
2.2.3	Ukážky výsledkov	8
2.3	Farebný filter.....	9
2.3.1	Postup riešenia	9
2.3.2	Použité metódy.....	9
2.3.3	Ukážky výsledkov	10
3	Vylepšenie kalibrácie	12
4	Záver	13

1 Zadanie

V tomto zadaní budete pracovať s 2D kamerou od spoločnosti Ximea. Cieľom je:

- realizovať kalibráciu kamery,
- aplikovať získané parametre na reálny obraz,
- implementovať detekciu geometrických tvarov,
- precvičiť si prácu s dokumentáciou OpenCV.

1.1 Úlohy

1. Kalibrácia kamery [2,5 b.]

Realizujte kalibráciu kamery pomocou šachovnice (chessboard pattern) podľa oficiálneho postupu v OpenCV.

Po kalibrácii:

- vypíšte maticu vnútorných parametrov kamery,
- určte hodnoty f_x , f_y , c_x , c_y ,
- uložte maticu kamery a distorzné koeficienty pre ďalšie použitie,
- demonštrujte odstránenie skreslenia (undistortion) na reálnom obraze.

Výsledkom musí byť funkčný program, ktorý vykoná kalibráciu z viacerých snímok šachovnice a vypočíta vnútorné parametre kamery.

2. Detekcia geometrických tvarov [2,5 b.]

Vytvorte program na detekciu základných geometrických tvarov (napr. kružnica, štvorec, trojuholník, obdĺžnik) v obraze pomocou vhodných metód dostupných v knižnici OpenCV (napr. Houghova transformácia, detekcia hrán, kontúry a ich aproximácia).

Program musí:

- detegovať tvary v reálnom obraze z kamery (Pomenovať),
- graficky označiť detegovaný tvar,
- zobrazit' stred (resp. ťažisko) tvaru,

Pri odovzdávaní demonštrujte detekciu na rôznych typoch tvarov (rôzna veľkosť, farba, osvetlenie, poloha v obraze).

3. Farebný filter [1,5 b.]

Implementujte farebný filter, ktorý:

- vyberie jednu konkrétnu farbu v obraze (napr. červenú),
- všetky pixely tejto farby nahradí inou zvolenou farbou.

Príklad: všetky červené objekty budú zobrazené ako zelené.

Farbu detegujte vhodným spôsobom (prahovanie vo vhodnom farebnom priestore). Výsledok musí fungovať v reálnom čase na obraze z kamery.

4. Dokumentácia [1,5 b.]

Vypracujte stručnú dokumentáciu vo formáte PDF, ktorá bude obsahovať:

- postup riešenia,
- použité metódy,
- ukážky výsledkov (obrázky),
- získané hodnoty parametrov kamery.

2 Vypracovanie

2.1 Kalibrácia kamery

2.1.1 Postup riešenia

Kalibráciu kamery sme realizovali pomocou sady fotografií šachovnice. V programe sme najskôr definovali kalibračný vzor ako šachovnicu s rozmerom 7×5 vnútorných rohov a veľkosťou jedného štvorca 0,024 m. Tieto údaje sme použili na vytvorenie 3D súradníc referenčných bodov šachovnice v rovine $Z = 0$.

Následne sme postupne načítali všetky obrázky z našej nafotenej sady. Každý obrázok bol prevedený do odtieňov sivej a na ňom sa pomocou funkcie *cv2.findChessboardCorners()* hľadali vnútorné rohy šachovnice. Po úspešnej detekcii boli ich polohy spresnené funkciou *cv2.cornerSubPix()*, čím sa zvýšila presnosť kalibrácie.

Zo všetkých úspešne spracovaných snímok sa vytvorili dve množiny bodov:

- **objpoints** – 3D body šachovnice v reálnom priestore,
- **imgpoints** – im prislúchajúce 2D body v obraze.

Po nazbieraní dostatočného počtu úspešných snímok bola zavolaná funkcia *cv2.calibrateCamera()*, ktorá vypočítala maticu kamery a distorzné koeficienty.

Na záver program uložil výsledky do súborov:

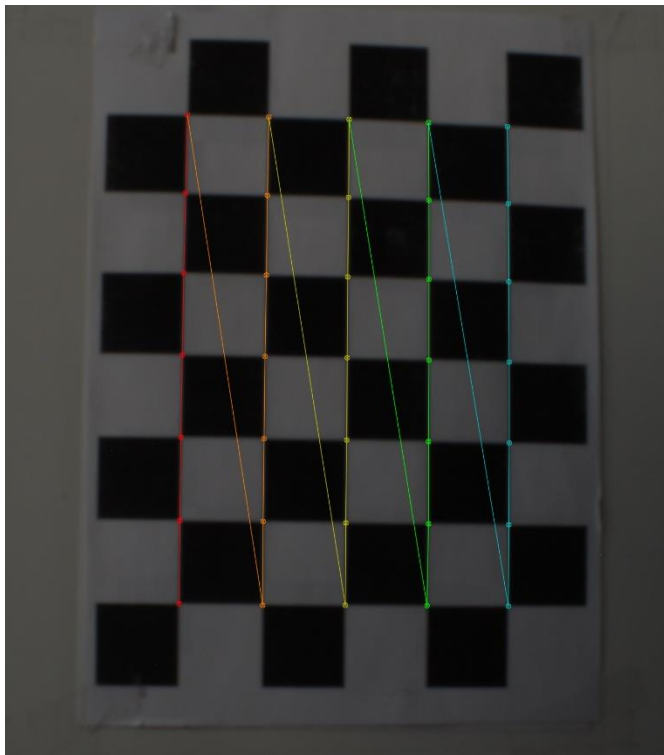
- calibration.npz
- calibration.yaml

a zároveň vytvoril aj ukážku odstránenia skreslenia obrazu pomocou *cv2.undistort()*.

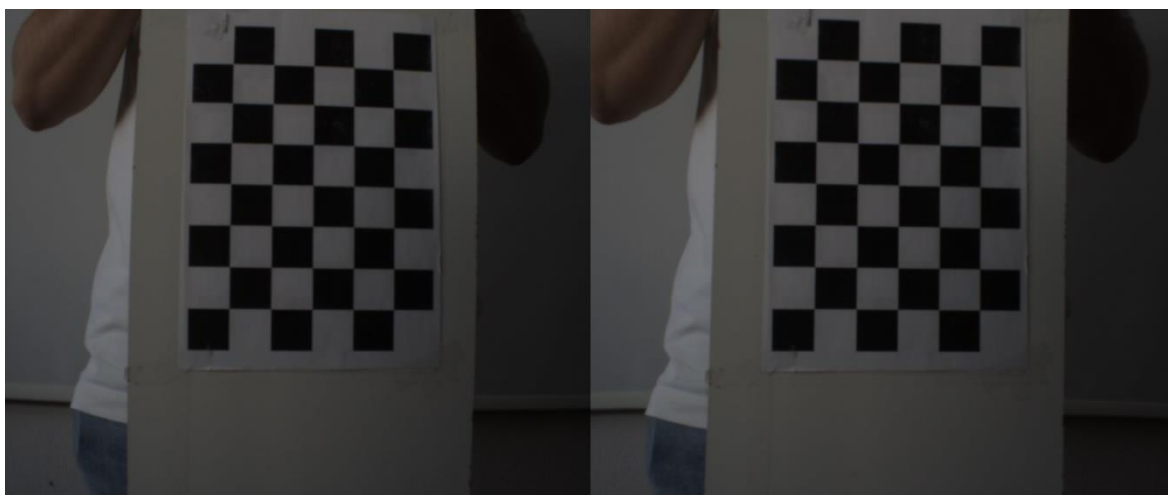
2.1.2 Použité metódy

- cv2.calibrateCamera()
- cv2.cornerSubPix()
- cv2.calibrateCamera()
- cv2.calibrateCamera()
- cv2.undistort()

2.1.3 Ukážky výsledkov



Obrázok 1 Detegované a spresnené vnútorné rohy šachovnice



Obrázok 2 Vľavo je fotka pred kalibráciou a vpravo po kalibrácii

2.1.4 Získaní hodnoty parametrov kamery

Matica vnútorných parametrov:

$$\begin{matrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{matrix}$$
$$\begin{matrix} 3640.54577 & 0 & 1270.20456 \\ 0 & 3638.33286 & 1037.14990 \\ 0 & 0 & 1 \end{matrix}$$

Distorzné koeficienty:

$$[-4.49415606\text{e-}01 \ 1.82927640 \ -1.15413348\text{e-}03 \ -1.63700944\text{e-}03 \ -1.10939414\text{e+}01]$$

2.2 Detekcia geometrických tvarov

2.2.1 Postup riešenia

Detekcia objektov sme neriešili iba pomocou hrán, ale pomocou segmentácie farebných objektov od pozadia. Keďže použité geometrické tvary boli vytvorené z farebných papierov na svetlej stene, bolo výhodné previesť obraz do farebného priestoru HSV a pracovať so zložkou saturácie. Farebné objekty majú vyššiu saturáciu než svetlé pozadie, preto bolo možné vytvoriť binárnu masku všetkých tvarov jednoduchým prahovaním. Po vytvorení binárnej masky sa maska vyčistila pomocou morfológických operácií, aby sa odstránil šum a spojili prípadne narušené časti objektov. Z takto upravenej masky sa vyhládali kontúry objektov.

Každá kontúra bola následne analyzovaná:

- spočítala sa jej plocha a obvod,
- aproximovala sa polygonálnou aproximáciou,
- vypočítalo sa ťažisko pomocou obrazových momentov,
- a podľa počtu vrcholov a ďalších parametrov sa určil typ tvaru.

Rozpoznávanie bolo realizované nasledovne:

- **3 vrcholy** → trojuholník,
- **4 vrcholy** → štvorec alebo obdĺžnik podľa pomeru strán,
- **viac ako 4 vrcholy** + dostatočná circularity → kružnica.

Pri kružnici sme na vykreslenie použili elipsu prispôbenú kontúre pomocou *cv2.fitEllipse()*.

Po rozpoznaní každého tvaru sme do obrazu vykreslili:

- obrys tvaru,

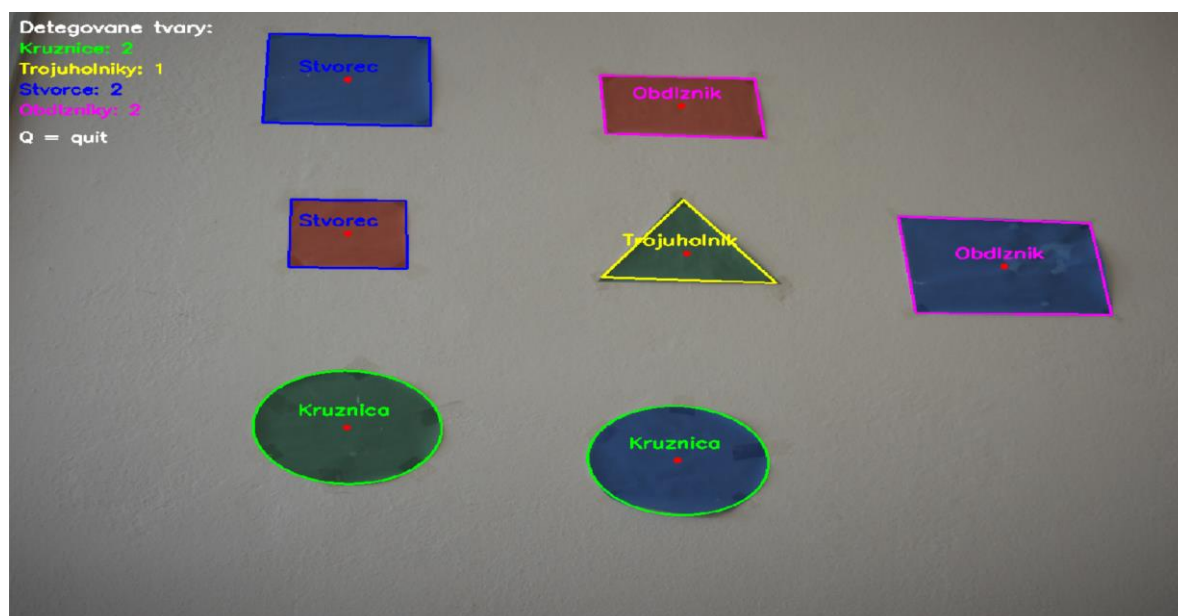
- názov tvaru,
- červená bodka predstavujúca jeho ťažisko.

V ľavom hornom rohu sa zároveň priebežne zobrazovali počty detegovaných objektov jednotlivých tried.

2.2.2 Použité metódy

- cv2.cvtColor()
- cv2.GaussianBlur()
- cv2.medianBlur()
- cv2.threshold()
- cv2.morphologyEx()
- cv2.findContours()
- cv2.approxPolyDP()
- cv2.moments()
- cv2.boundingRect()
- cv2.fitEllipse()
- cv2.drawContours()
- cv2.circle()

2.2.3 Ukážky výsledkov



Obrázok 3 Výstup pri live detegovaní tvarov z kamery

2.3 Farebný filter

2.3.1 Postup riešenia

Obraz sme previedli z farebného priestoru BGR do priestoru HSV, ktorý je vhodnejší na detekciu farieb. Červená farba sa v HSV priestore nachádza na začiatku aj na konci rozsahu zložky Hue, preto bolo potrebné použiť dva intervaly prahovania:

- prvý interval pre nižšie hodnoty červenej,
- druhý interval pre vyššie hodnoty červenej.

Pre oba intervaly sa vytvorili samostatné binárne masky, ktoré sa potom spojili logickou operáciou OR. Takto vznikla výsledná maska všetkých červených oblastí v obraze.

V ďalšom kroku sa vytvorila kópia pôvodného obrazu a všetky pixely, ktoré patrili do masky červenej farby, boli prepísané na zelenú farbu.

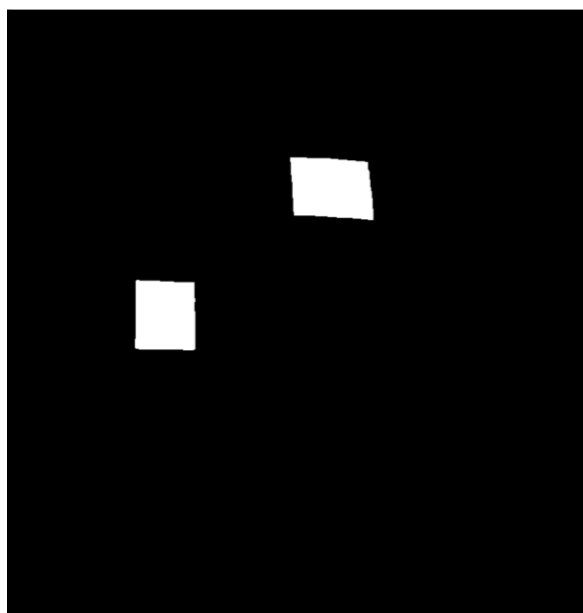
2.3.2 Použité metódy

- `cv2.cvtColor()`
- `cv2.inRange()`
- `cv2.bitwise_or()`
- `cv2.morphologyEx()`

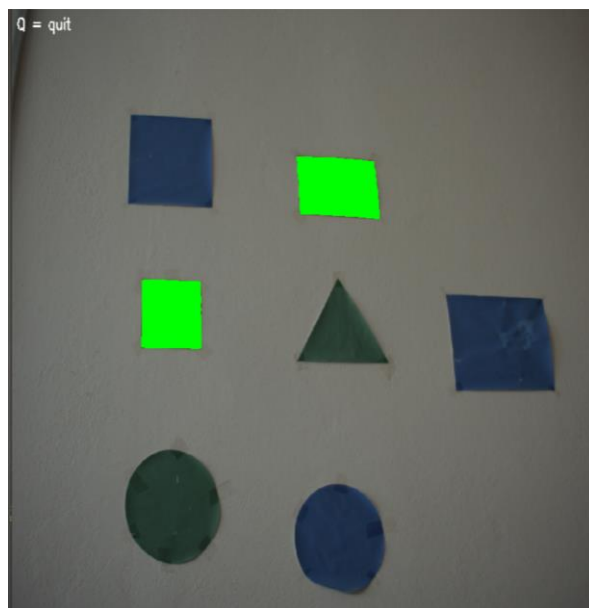
2.3.3 Ukážky výsledkov



Obrázok 4 Pôvodný obraz

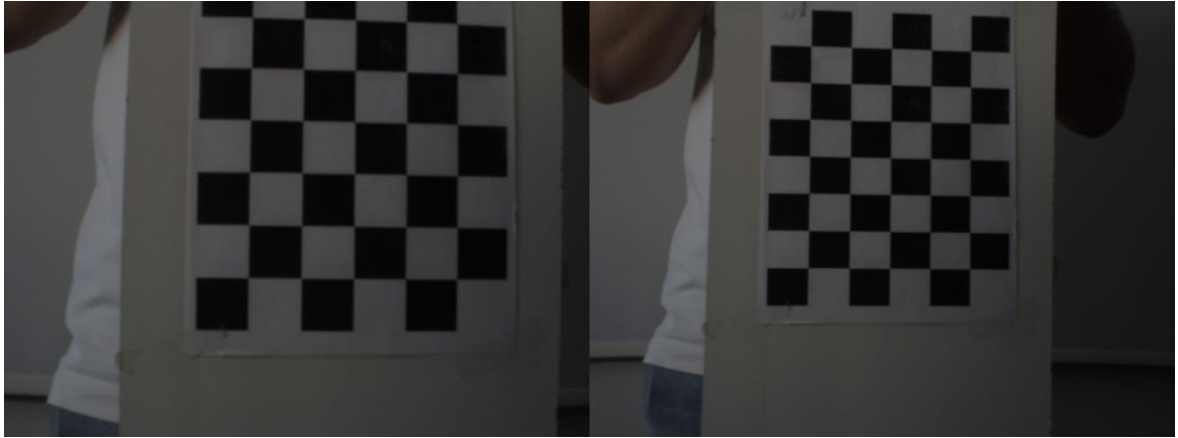


Obrázok 5 Maska červenej farby

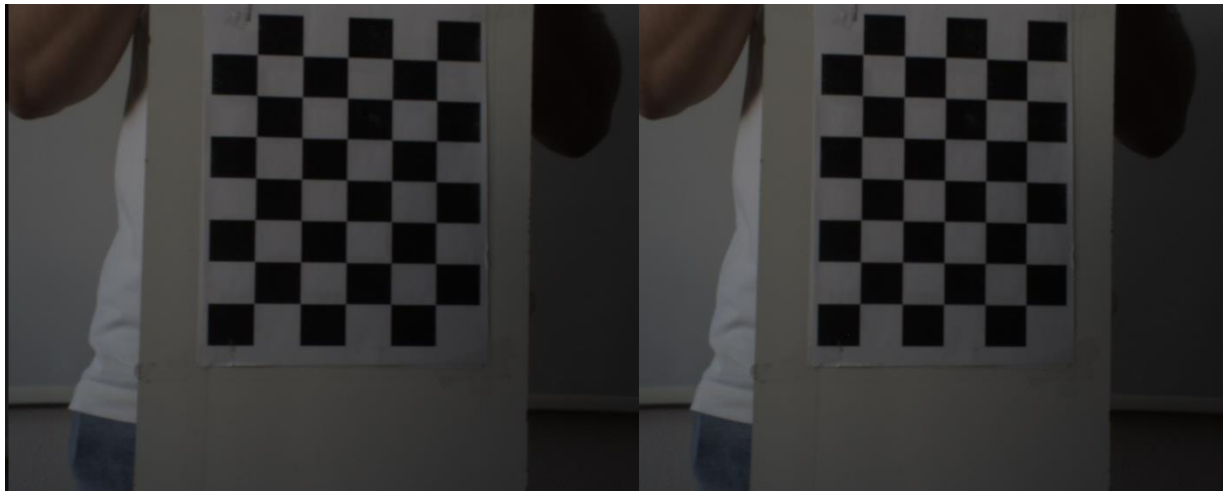


Obrázok 6 Výsledný obraz po nahradení červenej za zelenú

3 Vylepšenie kalibrácie



Obrázok 7 Vľavo je fotka pred kalibráciou a vpravo po kalibrácii (s využitím `cv.getOptimalNewCameraMatrix`)



Obrázok 8 Vľavo je fotka bez a vpravo s využitím `cv.getOptimalNewCameraMatrix`

4 Záver

V tomto zadaní sme pracovali s kamerou Ximea a knižnicou OpenCV. Úspešne sme naka-librovali kameru pomocou šachovnice, čím sme získali jej vnútorné parametre a odstránili skreslenie obrazu. Pri odovzdaní sme si našli nedostatok. Doplnili sme funkciu z OpenCV *cv.getOptimalNewCameraMatrix* ktorou sme naše vypracované zadanie ešte zdokonalili.

Implementovali sme detekciu geometrických tvarov, tento program v reálnom čase rozpoznáva a označuje kružnice, trojuholníky, štvorce a obdĺžniky v obraze.

Nakoniec sme vytvorili farebný filter, ktorý v reálnom čase nahrádza červenú farbu (spektrum) zelenou.